
pyMOR School 2026

Data-Driven Methods for Parametric Problems

Exercise Problems

Problem 1 (Selection criteria in VKOGA)

Compare the centers selected by the different selection criteria ('f', 'fp' and 'p') in the greedy of VKOGA for data generated by the function $f(x) = e^{-100(x-0.3)^2} + 0.2x$ on a uniform grid on the interval $[0, 1]$ when setting `max_centers=10`. Use a `GaussianKernel` with `length_scale=0.1`. What do you observe?

As in the lecture notebook, the selected centers are available after calling `fit` by accessing `regressor._surrogate._centers`, their indices in the training data are further given by the variable `regressor._surrogate._centers_idx`. Repeat the comparison for a second function of your choice and check how the selected centers change.

Problem 2 (Setting of the neural network-based regressor)

In the `NeuralNetworkRegressor`, try different settings for the neural network training, such as different optimizers. Adjust also the activation function and the architecture of the network. Find the default settings and investigate which parameters can be changed. Use the `NeuralNetworkRegressor` to approximate the same function as with VKOGA in Problem 1. Compare both resulting approximations.

You find all details on the usage of the `NeuralNetworkRegressor` and the `FullyConnectedNN` in the documentation.

Problem 3 (Using `scikit-learn` in pyMOR)

Use a regressor from `scikit-learn`, such as the `GaussianProcessRegressor`, as machine learning surrogate within the `DataDrivenReductor`.

Problem 4 (Anisotropic kernels for instationary problems)

For GPR, `scikit-learn` performs automatic hyperparameter tuning that can result in anisotropic kernels. Apply GPR in the case of an instationary problem and compare the setting `time_vectorized=True` to `time_vectorized=False`. Investigate the optimized kernel. What do you observe?

Observe that the two settings require different kernels: for `time_vectorized=False` the regressor receives the parameter *and* the time instance as input, such that the kernel can use

one length scale per input direction, whereas for `time_vectorized=True` only the parameter is passed and an isotropic kernel has to be used:

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C

kernel_random_access = C(1.0, (1e-3, 1e3)) * RBF([1.0, 1.0], (1e-2, 1e2))
kernel_time_vectorized = C(1.0, (1e-3, 1e3)) * RBF(1.0, (1e-2, 1e2))

gpr_regressor = GaussianProcessRegressor(kernel=kernel_random_access,
                                         normalize_y=True, random_state=0)
```

You can then access the trained kernel after calling `reduce` as `gpr_rom.regressor.kernel_`. To find out which of the two length scales belongs to the time direction, take a look at `mu.at_time(t).to_numpy()`, which orders the entries by the (alphabetically sorted) names of the parameters.

Problem 5 (Model hierarchy with two models)

Build a model hierarchy containing only the FOM and the RB-ROM and analyze its performance using the `two_dimensional_parametric_diffusion` example from the lecture notebook.

Problem 6 (The model hierarchy in a multi-query scenario)

Apply the adaptive model hierarchy for uncertainty quantification by solving the lecture notebook's `two_dimensional_parametric_diffusion` problem for 1000 randomly sampled parameters and estimate the mean and variance of the output using unbiased estimators (pick one of the two output components). Plot the evolution of the estimates over the number of sampled parameters. Do you observe convergence? Can you provide an error estimate for the output estimators that takes the tolerance, used in the hierarchy, into account?

The required quantities are returned by the hierarchy via

```
data = hierarchy.compute(output=True, output_error_estimate=True, mu=mu)
```

For the error estimate, keep in mind that the tolerance bounds the error of every single returned solution, whereas the Monte Carlo estimator introduces an additional error that decreases with the number of samples.

Problem 7 (Approximating datasets using VKOGA)

Use the `VKOGARegressor` to approximate a different dataset, for instance from Hugging Face. Compare the result to a linear regression and vary `length_scale` and `max_centers`. Is a larger number of centers always better?

Datasets from Hugging Face require the additional package `datasets`, which can be made available for a single session by running `uv run --with datasets jupyter lab`. The following snippet loads a regression dataset with 8192 samples and 21 features, subsamples it to keep the greedy fast and scales inputs and outputs:

```
import numpy as np
from datasets import load_dataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler

dataset = load_dataset('inria-soda/tabular-benchmark', 'reg_num_cpu_act',
                       split='train').to_pandas()
targets = dataset.pop('usr').to_numpy(float)  # CPU time in user mode
features = dataset.to_numpy(float)

rng = np.random.default_rng(0)
indices = rng.choice(len(features), 2000, replace=False)
X_train, X_test, y_train, y_test = train_test_split(features[indices],
                                                    targets[indices],
                                                    test_size=0.2)

input_scaler = MinMaxScaler().fit(X_train)
output_scaler = StandardScaler().fit(y_train[:, None])

regressor = VKOGARegressor(kernel=GaussianKernel(length_scale=2.),
                           max_centers=50)
regressor.fit(input_scaler.transform(X_train),
              output_scaler.transform(y_train[:, None]))
prediction = output_scaler.inverse_transform(
    regressor.predict(input_scaler.transform(X_test)))
```